Real-world (rebuild Doc04's extraction, LCEL-style) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `lcel_vs_raw_sdk_practice.py`: reading about "LangChain vs. raw SDK" isn't the same as running both versions of one real feature side by side. This rebuilds Doc04's `get_structured_reply()` with LCEL and compares the two. **If not:** the Build Task's `compare_with_raw_sdk.py` would be the first time you built this comparison, with no smaller version to trust it against.

Solution

Story — `compare()`: the script decides "match" itself, with a fair rule: names must agree; intent wording may differ. **If not:** a human reads printed text and misses a real difference.

```
# lcel_vs_raw_sdk_practice.py
from dotenv import load_dotenv
from openai import OpenAI
from pydantic import BaseModel
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable
from langchain_openai import ChatOpenAI

load_dotenv()

# Doc04's prompt, schema, and raw-SDK helper, inline so this file runs
# on its own.
EXTRACT_PROMPT = (
    "Extract the person's name and what they want. "
    "Return the name and a short intent."
)

class ExtractedInfo(BaseModel):
    name: str
    intent: str

def create_client() -> OpenAI:
    return OpenAI()
```

```
def get_structured_reply(client: OpenAI, text: str) -> ExtractedInfo:
    completion = client.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": EXTRACT_PROMPT},
            {"role": "user", "content": text},
        ],
        response_format=ExtractedInfo,
    )
    return completion.choices[0].message.parsed

# how: every input has a name, so neither version returns None here
TEST_INPUTS = [
    "Maria wants to reset her password",
    "Hi, I'm Tom and my invoice looks wrong",
    "Ahmed here, please cancel my order",
]

def build_lcel_chain(model_name: str = "gpt-4o-mini") -> Runnable:
    prompt = ChatPromptTemplate.from_messages(
        [("system", EXTRACT_PROMPT), ("human", "{text}")]
    )
    model = ChatOpenAI(model=model_name, temperature=0)
    return prompt | model.with_structured_output(ExtractedInfo)

def compare(client: OpenAI, lcel_chain: Runnable, text: str) -> bool:
    raw_result = get_structured_reply(client, text)
    lcel_result = lcel_chain.invoke({"text": text})
    # why: a name has one right answer, so both versions must agree on
    # it; intent is free wording that can differ between any two calls
    matched = raw_result.name == lcel_result.name
    print(f"{text} -> match: {matched}")
    print(f"  raw:  {raw_result}")
    print(f"  lcel: {lcel_result}")
    return matched

def main() -> None:
    client = create_client()
    lcel_chain = build_lcel_chain()
    matched = 0
    for text in TEST_INPUTS:
        if compare(client, lcel_chain, text):
            matched = matched + 1
    print(f"{matched}/{len(TEST_INPUTS)} matched")
```

```
if __name__ == "__main__":
    main()
```

Expected output (wording varies):

```
Maria wants to reset her password -> match: True
  raw:  name='Maria' intent='reset her password'
  lcel: name='Maria' intent='reset her password'
Hi, I'm Tom and my invoice looks wrong -> match: True
  raw:  name='Tom' intent='check a wrong invoice'
  lcel: name='Tom' intent='check a wrong invoice'
Ahmed here, please cancel my order -> match: True
  raw:  name='Ahmed' intent='cancel his order'
  lcel: name='Ahmed' intent='cancel his order'
3/3 matched
```